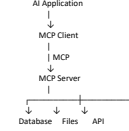




So a simple analogy is:
USB standardizes device connectivity; MCP standardizes AI application connectivity to external tools and data.
It's an analogy—the actual MCP protocol is more specific than USB.

4. HOW does MCP work?

The basic architecture is:

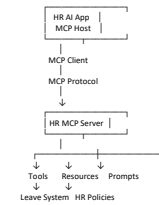


There are three important components.

MCP Host
The AI application itself.
For example:
AI Agent
AI Assistant
IDE
AI Application

MCP Client
The component that communicates with the MCP server.
AI Application
↓
MCP Client
↓
MCP
↓
MCP Server
↓
Database Files API

MCP Server
The server that exposes capabilities to the AI application.
For example:
HR MCP Server
Tools:
get_employee()
get_leave_balance()
search_policy()
Resources:
leave_policy.pdf
employee_data



Now the user asks:
"How many leaves does E102 have?"
The flow is:
User
↓
HR AI App
↓
MCP Client
↓
MCP Server
↓
get_leave_balance("E102")
↓
Leave Database
↓
12
↓
MCP Server
↓
MCP Client
↓
AI App
↓
"E102 has 12 leaves."

9. Where JSON Schema fits

You just covered JSON Schema, so connect it here.
The MCP Server exposes a tool along with its input schema.
For example:

```
{
  "name": "get_leave_balance",
  "description": "Get employee leave balance",
  "inputSchema": {
    "type": "object",
    "properties": {
      "employee_id": {
        "type": "string"
      }
    },
    "required": ["employee_id"]
  }
}
```

The AI application now knows:
Tool:
get_leave_balance
Input:
employee_id
Type:
string
Required:
yes
So it can make the correct tool call.

10. MCP communication

At a high level:



For example:
Client:
"What tools are available?"
↓

Server:
"get_employee
get_leave_balance
search_policy"
↓

Client:
"Call get_leave_balance
employee_id = E102"
↓

Server:
"12"
The actual MCP protocol uses structured messages and defines the lifecycle and operations for this interaction.

11. MCP vs Agent-to-Agent

This distinction is especially important because your larger topic includes agent communication.

MCP



Agent -> Tool/Data

Agent-to-Agent



Agent -> Agent

So don't describe MCP as simply an "agent-to-agent protocol."

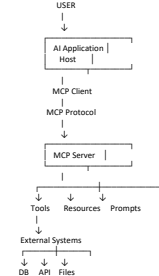
A useful architecture is:



Here:
• **Agent-to-agent communication** connects the agents.
• **MCP** connects each agent/application to tools and resources.

12. The architecture in one picture

This is the diagram I would use as your main slide:



The 5 things to remember

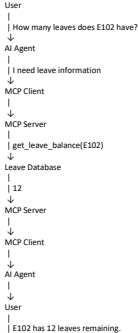
Host

6. Real example — HR system

Suppose we have:



The user asks:
"How many leaves does E102 have?"
The flow becomes:



This is the basic idea behind MCP.

7. MCP is NOT the LLM

This distinction is important.



The LLM/agent reasons about what needs to be done.
MCP provides the **standard communication mechanism** for accessing external capabilities.

For example:
LLM:
"I need E102's leave balance."
↓
MCP:
get_leave_balance(employee_id="E102")
↓

HR Database:
12 leaves
↓
LLM:
"E102 has 12 leaves remaining."

8. MCP does NOT replace REST APIs

This is another important concept.

MCP and REST APIs serve different purposes.

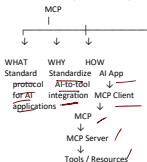


The MCP server can internally communicate with existing:

- REST APIs
- Databases
- Python functions
- Files
- Cloud services

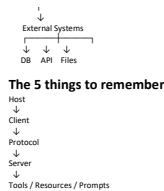
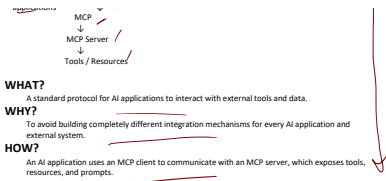
So you don't need to throw away your existing APIs just because you introduce MCP.

9. WHAT → WHY → HOW in one view



WHAT?

A standard protocol for AI applications to interact with external tools and data.

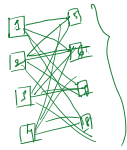
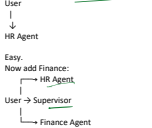


Build Scalable Agent Communication Networks

The basic idea is:
When the number of agents increases, we need an architecture that allows agents to communicate efficiently without creating a separate direct connection between every pair of agents.

1. Start with a simple system

Suppose we have three agents:



The number of communication paths starts increasing.

2. The problem with direct communication

Imagine 5 agents all communicating directly:



With N agents, the number of possible direct connections can grow roughly as:

$$N \times (N-1) / 2$$

For example:

Agents	Possible direct connections
2	1
3	3
4	6
5	10
6	15
7	21
8	28
9	36
10	45
20	190
50	1,225

So a fully connected architecture becomes difficult to manage.

3. Solution: Supervisor / Orchestrator

Instead of every agent talking directly to every other agent:



use a central orchestrator:



The Supervisor decides:
Which agent should handle this task?

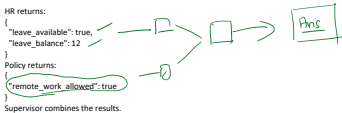
4. Example

User asks:
"Can I take leave and work remotely for those days?"

Supervisor determines
This requires:

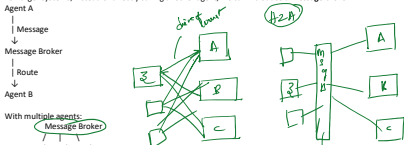
HR Agent
Policy Agent

So:

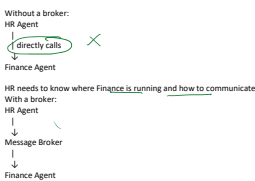


5. Message-based architecture

For larger systems, instead of directly calling another agent, we can introduce a message broker.



6. Why use a message broker?

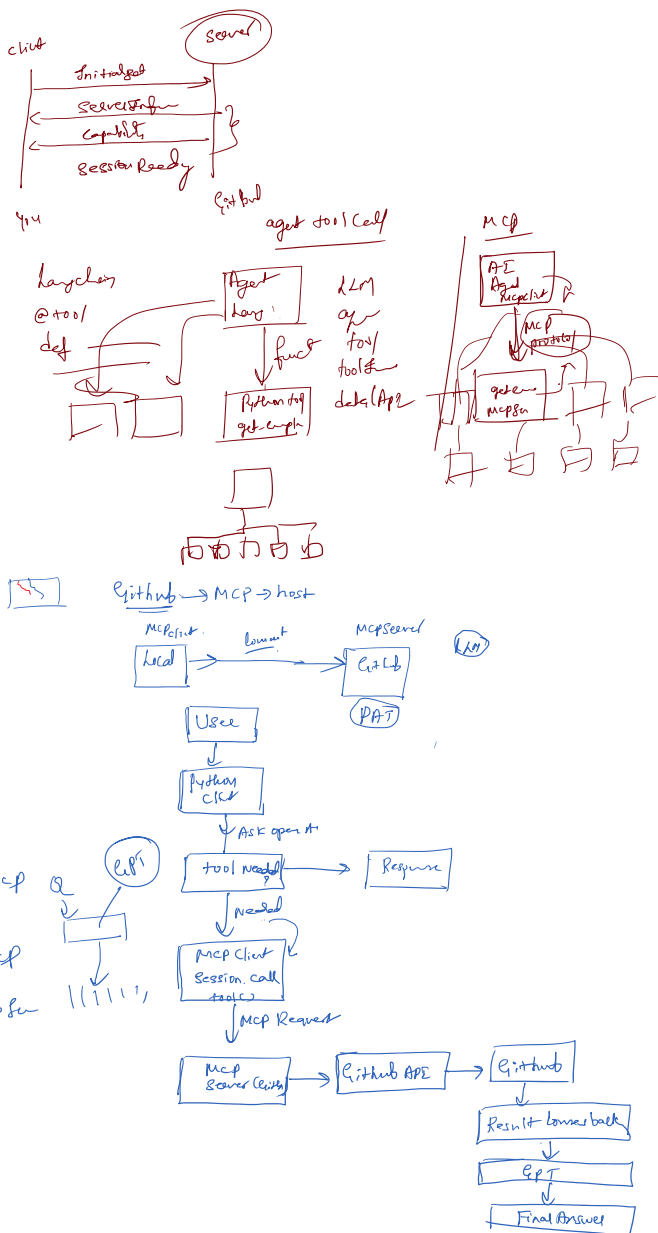


HR doesn't need to know the exact implementation of Finance Agent.

7. Example message

Suppose HR Agent wants Finance Agent to calculate salary impact.

{



↓
Finance Agent
HR doesn't need to know the exact implementation of Finance Agent.

7. Example message

Suppose HR Agent wants Finance Agent to calculate salary impact.

```
{
  "message_id": "msg-101",
  "sender": "hr-agent",
  "receiver": "finance-agent",
  "message_type": "task",
  "task": "calculate_leave_salary_impact",
  "context": {
    "employee_id": "E102",
    "leave_days": 5
  }
}
```

The broker routes this message to Finance Agent.

Finance returns:

```
{
  "message_id": "msg-102",
  "sender": "finance-agent",
  "receiver": "hr-agent",
  "message_type": "response",
  "status": "success",
  "result": {
    "salary_impact": 2500
  }
}
```

8. Synchronous vs asynchronous communication

This is important for scalable systems.

Synchronous

Agent waits for the response.

HR Agent

↓ request

Finance Agent

↓ response

HR Agent

Useful when the result is immediately required.

Asynchronous

Agent sends the request and continues working.

HR Agent

↓ request

Message Broker

↓

Finance Agent

HR Agent continues working...

Later:

Finance Agent

↓ result

Message Broker

↓

HR Agent

This is useful for long-running tasks.

9. Add agent discovery

In a large system, the Supervisor needs to know:

Which agent can handle this task?

Maintain an agent registry:

Agent Registry

HR Agent

capabilities:

- leave

- employee information

Finance Agent

capabilities:

- salary

- expenses

IT Agent

capabilities:

- laptop

- access

- software

Legal Agent

capabilities:

- contracts

- compliance

Then:

Supervisor

↓

Agent Registry

↓

HR Agent

This makes the network more dynamic.

10. Scalable architecture

A more complete architecture looks like:

USER

↓

Supervisor Agent

↓

Agent Registry

↓

Message Broker

↓

HR Agent Finance Agent IT Agent

↓

MCP Server MCP Server MCP Server

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

HR DB Finance DB IT Systems

↓

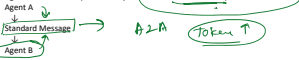
HR DB Finance DB IT Systems

2. HOW TO MAKE THE NETWORK ROBUST

There are several important design principles:

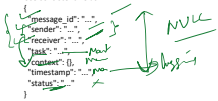
1. Loose coupling

Agents should not depend heavily on each other's internal implementation.



2. Standard message format

Use consistent fields:



3. Agent discovery

Maintain:
Agent Registry
so agents can be located based on their capabilities.

4. Message broker

Use a messaging layer when communication becomes large or asynchronous.

Examples include:

- RabbitMQ
- Redis Streams
- Cloud Pub/Sub

5. Failure handling

What happens if Finance Agent is unavailable?

Supervisor

↓

Finance Agent

X

Failed

↓

Retry / Timeout / Fallback

The system should support:

- Timeout
- Retry
- Error response
- Fallback agent
- Queue-level queue

6. Observability

Track:

message_id

agent

task

latency

status

error

For example:

msg-101

Supervisor → HR

Status: SUCCESS

Time: 420 ms

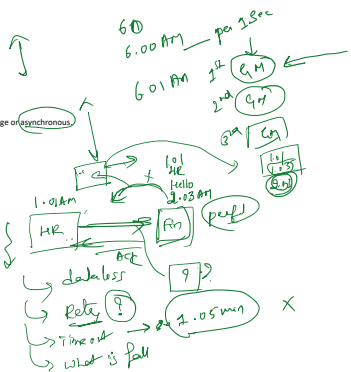
msg-102

Supervisor → Finance

Status: FAILED

Retry: 2

This becomes extremely important as the number of agents grows.



MCP SDK

1. What is the MCP SDK?

MCP SDK = Software Development Kit for building MCP clients and MCP servers

Instead of manually implementing the MCP protocol, the SDK provides ready-made classes, functions, and utilities.

Conceptually:

Without SDK

Your Python Code

↓

Implement Protocol

↓

Handle Messages

↓

Handle Tools

↓

Handle Resources

↓

Handle Communication

↓

With an SDK:

Your Python Code

↓

MCP SDK

↓

MCP Server / MCP Client

So:

The MCP SDK makes it easier to build applications that communicate using the MCP protocol.

2. What can you build with the MCP SDK?

Mainly two sides:

MCP

↓

MCP Server

↓

MCP Client

↓

Exposes tools

↓

Connects to

MCP servers

↓

MCP Server

↓

You can create a server that exposes:

Tools

Resources

Prompts

Example:

HR MCP Server

Tools:

get_employee()

get_leave_balance()

apply_leave()

Resources:

leave_policy

employee_policy

MCP Client

↓

You can create an application that connects to the MCP server and discovers/calls those capabilities.

3. Python MCP SDK

For Python, the commonly used official SDK package is:

pip install mcp

Then you can build an MCP server.

A minimal example:

from mcp.server.fastmcp import FastMCP

mcp = FastMCP("HR Server")

@mcp.tool()

def get_leave_balance(employee_id: str) -> int:

"""Return the leave balance for an employee."""

leave_data = {

"E101": 10,

"E102": 15,

"E103": 8

}

return leave_data.get(employee_id, 0)

if __name__ == "__main__":

mcp.run()

The important part is:

@mcp.tool()

This tells the MCP server:

"Expose this Python function as an MCP tool."

4. What happens here?

You write an ordinary Python function:

def get_leave_balance(employee_id: str):

The MCP SDK turns it into an MCP tool:

Python Function

↓

@mcp.tool()

↓

MCP Tool

↓

AI Application

def get_leave_balance(employee_id:str):
The MCP SDK turns it into an MCP tool:
Python Function
↓
@mcp.tool()
↓
MCP Tool
↓
AI Application
The AI application can discover that the server provides:
get_leave_balance
with an input:
employee_id: string
User

7. MCP SDK vs MCP

This distinction is important.
MCP
The protocol/specification.
It defines how communication should work.
MCP = Rules
MCP SDK
The developer toolkit used to implement those rules.
SDK = Tools for implementing the rules
A simple analogy:
HTTP
↓
Protocol
Requests library
↓
Software library for using HTTP
Similarly:
MCP
↓
Protocol
MCP SDK
↓
Software toolkit for building MCP applications

Secure State Synchronization

This fits naturally after context sharing and scalable agent communication.
The core idea is:
Secure state synchronization means keeping the important state of multiple agents consistent while ensuring that only authorized agents can access or modify it.

1. What is "state"?

State is information that represents the current situation of a task or system.
For example, in an HR multi-agent system:
{
"task_id": "TASK-101",
"employee_id": "E102",
"leave_requested": 5,
"leave_balance": 12,
"manager_approved": true,
"status": "pending"
}

This is the current state of the task.
Multiple agents may need some of this information:



2. What is synchronization?

Suppose the HR Agent updates:
leave_balance = 7
But the Supervisor still has:
leave_balance = 12
Now we have inconsistent state.
HR Agent
↓
Balance = 7
Supervisor
↓
Balance = 12

Synchronization means making sure the relevant agents have the correct/latest state.

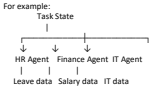


3. Why is security needed?

Imagine an employee's salary information is part of the state.
You don't want:
IT Agent
↓
Access
↓
Employee Salary
If the IT Agent doesn't have permission.
So synchronization needs both:
State synchronization
↓
Security

4. What should be protected?

Important state might include:
Employee information
Leave information
Salary information
Task status
Agent credentials
Tool results
Conversation context
Approval information
Not every agent should see everything.
For example:



Each agent gets only the information it needs.

This is the principle of:
Least privilege

5. Authentication

Before an agent communicates with another service, the system should verify:

Who are you?

Conceptually:

Agent

↓

Credentials / Token

↓

Authorization Service

↓

Valid?

↓

Access Granted

Common mechanisms include:

• API keys

• OAuth 2.0

• JWT

• mTLS

• Service identities

The exact mechanism depends on the architecture.

6. Authorization

Authentication answers:

Who are you?

Authorization answers:

What are you allowed to do?

Example:

HR Agent

↓

Can read:

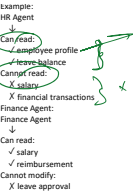
• employee profile

• leave balance

Cannot read:

• salary

• financial transactions



7. Secure state structure

A state object can include metadata such as:

```
{
  "task_id": "TASK-101",
  "version": 4,
  "updated_by": "hr-agent",
  "timestamp": "2026-08-13T10:30:00Z",
  "data": {
    "employee_id": "E102",
    "leave_balance": 7
  }
}
```

The important fields are:
task_id
↓
Which task?

version
↓
Which version of state?

updated_by
↓
Which agent changed it?

timestamp
↓
When was it changed?

data
↓
Actual state

This prevents an old state from accidentally overwriting newer state.

10. State synchronization using events

Another approach is event-based synchronization.
Instead of constantly sending the entire state:
Agent → Entire State → Agent

send changes as events:

```
{
  "event": "LEAVE_APPROVED",
  "task_id": "TASK-101",
  "employee_id": "E102",
  "days": 5,
  "timestamp": "2026-08-13T10:30:00Z"
}
```

Other agents consume the event:



This can scale much better for large systems.

11. Encryption

State can contain sensitive information.
Use encryption:

In transit



Typically using secure transport such as TLS.

At rest



So even if someone obtains the stored data, it isn't simply readable.

12. Audit trail

For multi-agent systems, you should know:

Who changed the state?

When?

What changed?

Why?

For example:

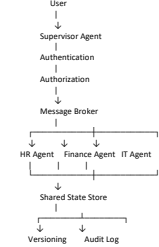
```
TASK-101
10-20 Supervisor created task
10-22 hr Agent checked leave
10-24 Manager approved leave
10-25 Policy Agent validated policy
10-26 Supervisor completed task
```

This is extremely useful for:

- Debugging
- Security
- Compliance
- Troubleshooting
- Accountability

13. Complete architecture

Putting everything together:



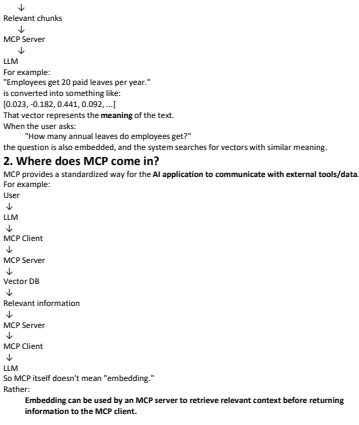
Security exists across the communication and state layers:

- Authentication
- +
- Authorization
- +
- Encryption
- +
- Versioning
- +
- Audit Logging

1. What is Embedding in MCP?

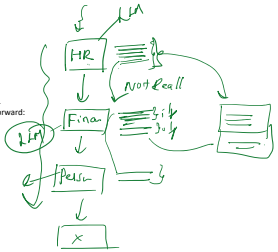
Suppose an MCP client asks:
"What is our leave policy?"
The MCP server may have thousands of documents.
Instead of sending all documents to the LLM, we can:
Company Documents





3. What is Context Propagation?

This is more directly related to MCP.
Imagine an agent is working on:
"Analyze employee Rahul's leave balance."
The first tool call gets employee information:
{
 "employee": "Rahul",
 "department": "IT",
 "employee_id": "E101"
}
Now another MCP tool needs to retrieve Rahul's leave information.
Instead of starting from zero, the relevant context can be carried forward:
Tool 1
Employee = **Rahul**
Department = **IT**
ID → E101
Context
↓
Tool 2
(Leave balance → 12 days)
This is **context propagation**.



4. Simple MCP Example

Imagine we have two MCP tools:
get_employee()
get_leave_balance()
User asks:
"How many leaves does Rahul from IT have?"
Step 1 — MCP calls employee tool
get_employee("Rahul")
Response:
{
 "employee_id": "E101",
 "name": "Rahul",
 "department": "IT"
}
Step 2 — Context is propagated
The next operation now knows:
employee_id = E101
So it can call:
get_leave_balance("E101")
Response:
{
 "employee_id": "E101",
 "leave_balance": 12
}
Step 3 — LLM produces answer
Rahul from the IT department has 12 leaves remaining.
The important idea is:
User Context
↓
MCP Client
↓
MCP Server
↓
Tool 1
↓
Context
↓
Tool 2
↓
LLM

5. Embedding + Context Propagation Together

This becomes particularly useful for MCP + RAG.
Example:
User:
"What is the leave policy for IT employees?"
↓
MCP Client
↓
MCP Server
↓
Create embedding
↓
Vector Search
↓
Relevant HR documents
↓
Context
↓
MCP response
↓
LLM
↓
"IT employees receive 20 annual leaves..."
Here:
Embedding answers:
"Which information is relevant?"
Context propagation answers:
"How do we carry the relevant information/context through the workflow?"